

OAuth Authentication Web App

- Group Members:

- Mohamed Ashraf Abbas (2206196)
- Nada Magdy (2206170)
- Aliaa Omar (2206187)
- Mohamed Ahmed Mohamed Kamel (22011683)
- Marwan Abdelhamid (2206174)
- Omar El-sherbiny (2206206)
- Mohamed Ezzat (2206166)

- Under The Supervision of:

- Dr. Hossam Selim
- Eng. Ahmed Osama

- Project Objective

simple web application that demonstrates **OAuth 2.0 authentication** using multiple identity providers (**GitHub** and **Google**). Users can log in with their preferred provider and are greeted with a customized success message. The app also includes a logout function and session status check.

- Team Roles

Member	Member Role	Responsibility
Mohamed Ezzat & Marwan Abdelhamid	Frontend Developer	Designed <code>index.html</code> and notification logic
Mohamed Ashraf & Nada magdy & Aliaa Omar & Mohamed Kamel	Backend Developer	Wrote <code>server.js</code> and handled OAuth routes
Mohamed Ashraf & Mohamed Kamel	API Integrator	Registered apps with Google/GitHub and tested tokens
Omar Elsherbiny	Debugger	Fixed issues with routes, tokens, and flows

- What Is OAuth?

OAuth 2.0 is an **authorization framework** that allows third-party applications to access a user's information **without sharing their password**. It is widely used in modern web applications.

- How OAuth Works:

1. **User clicks "Login with X"** → the app redirects them to the identity provider (e.g., GitHub or Google).
2. **User logs in** and authorizes the app.
3. The provider **redirects back** with an **authorization code**.
4. The app uses the code to request an **access token** from the provider.
5. The app stores the token (or session) and considers the user authenticated.
6. **User can logout**, clearing the session/token.

This ensures secure authentication **without the app ever seeing the user's credentials**.

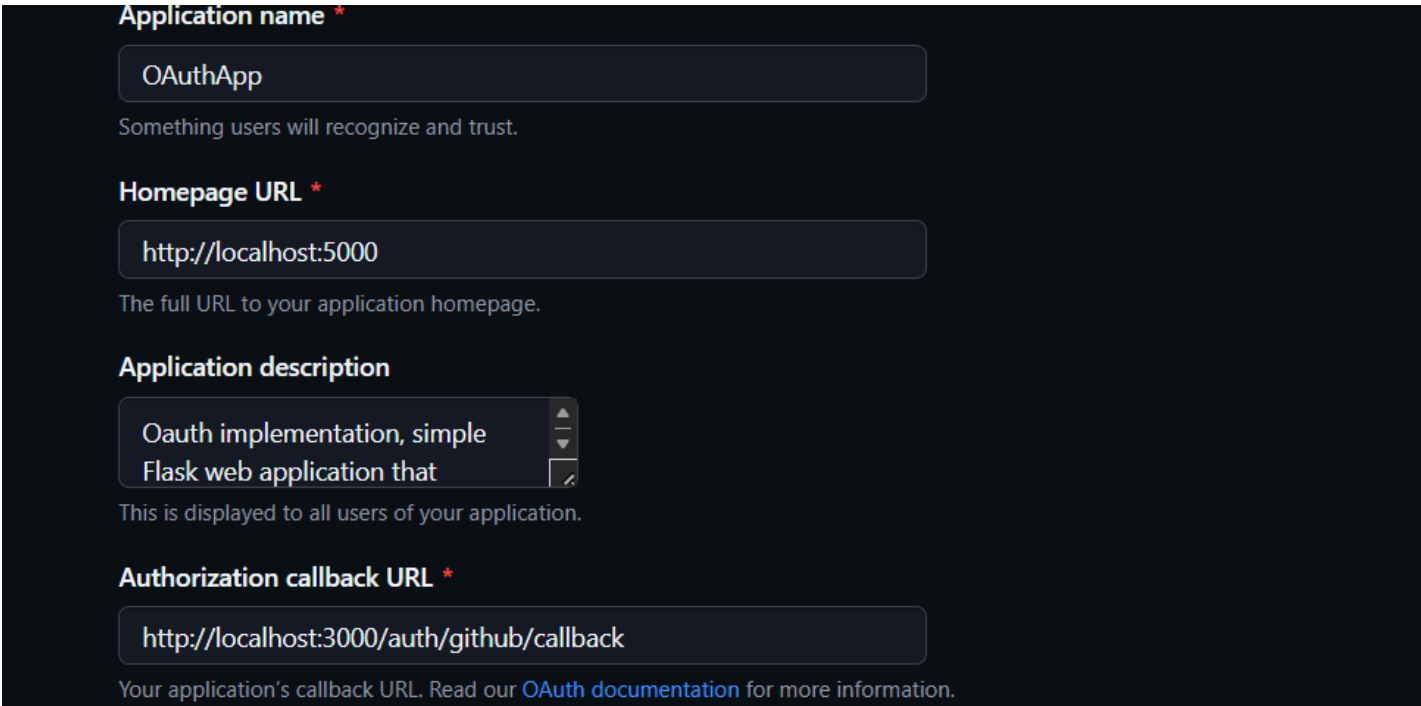
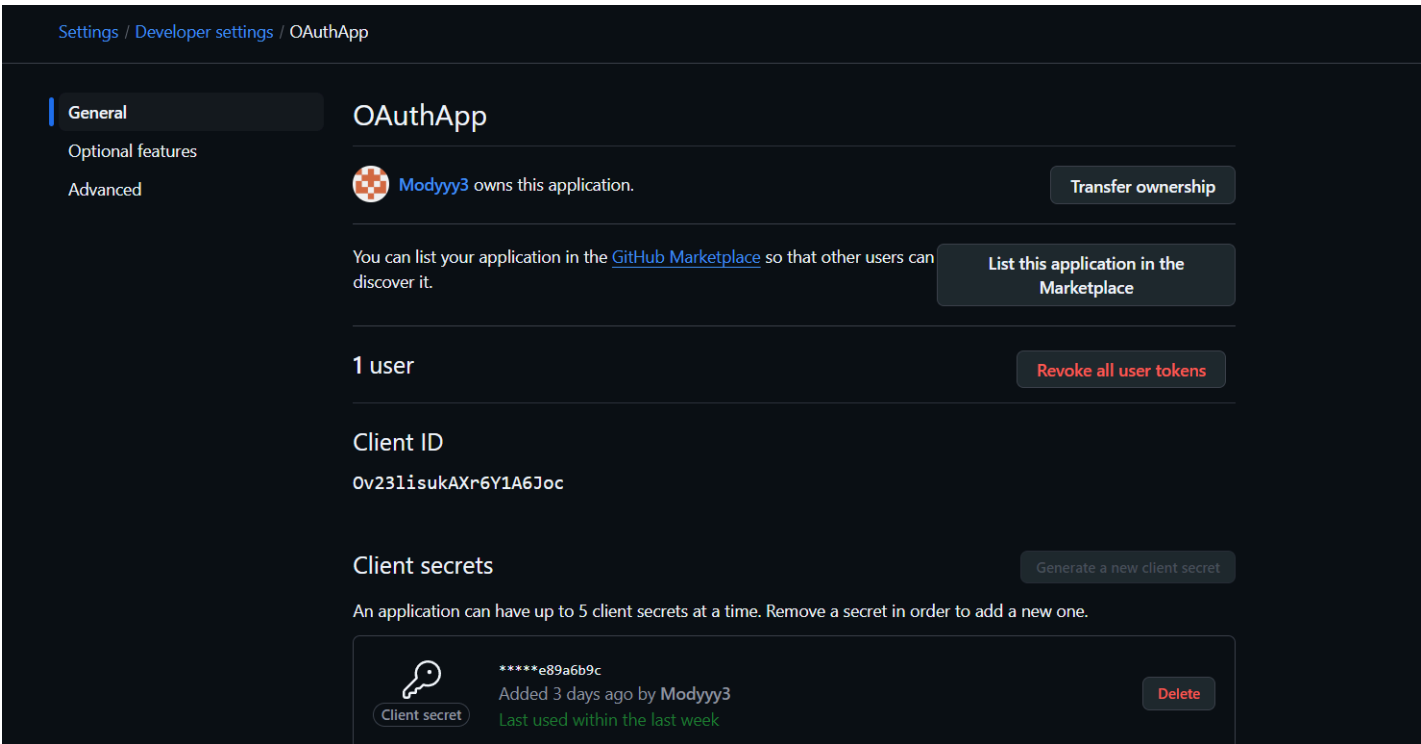
- Technologies Used

- **Node.js & Express** — Backend server
- **Axios** — To send HTTP requests to OAuth token endpoints
- **HTML/CSS/JavaScript** — Frontend layout and notifications
- **dotenv** — Manage sensitive API keys
- **GitHub & Google** — Identity providers

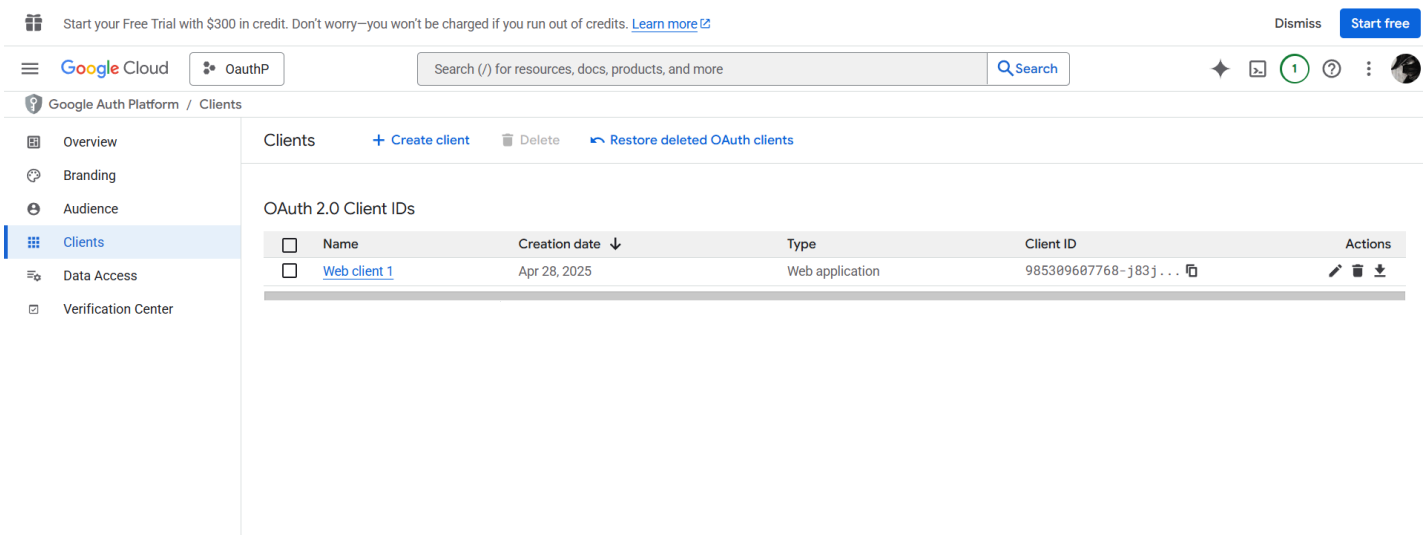
Step One (Setting The API's)

- Setting up and managing third-party authentication providers (like Google and GitHub).
- Registering the app on each provider's developer console.
- Getting and securing **Client ID** and **Client Secret**.
- Ensuring the correct **redirect URLs** are used.
- Testing and validating the token exchange process.

- Github



- Google



Additional information

Client ID	985309607768-j83jjj1sr1s87637753ffmv42664lhg3.apps.googleusercontent.com
Creation date	April 28, 2025 at 1:57:08 AM GMT+3

Client secrets

If you are in the process of changing client secrets, you can manually rotate them without downtime. [Learn more](#)

Client secret	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
Creation date	April 28, 2025 at 1:57:08 AM GMT+3
Status	Enabled

Step two (Frontend)

(**index.html**)

- File: `public/index.html`
- This is the main user interface.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>OAuth App</title>
  <script>
    // As soon as the page loads, check for "?login=success"
    document.addEventListener('DOMContentLoaded', async () => {
      const params = new URLSearchParams(window.location.search);
      if (params.get('login') === 'success') {
        // Get the provider from the session
        try {
          const response = await fetch('/session');
          const session = await response.json();

          if (session.provider) {
            const message = `Login successful with ${session.provider}!`;

            // 1) Show browser notification if permission granted
            if (Notification.permission === 'granted') {
              new Notification(message);
            } else if (Notification.permission !== 'denied') {
              Notification.requestPermission().then(permission => {
                if (permission === 'granted') {
                  new Notification(message);
                }
              });
            }

            // 2) Show in-page alert
            alert(message);
          }

          // 3) Clean up the URL (remove the ?login=success)
          params.delete('login');
          history.replaceState(null, "", window.location.pathname);
        } catch (error) {
          console.error('Error checking session:', error);
        }
      }
    });
  </script>
  <style>
    body {
      background-color: #191970;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
```

```

    color: white;
    font-family: Arial, sans-serif;
    text-align: center;
  }
  .box {
    background: white;
    color: black;
    padding: 30px;
    border-radius: 10px;
  }
  button {
    margin: 10px;
    padding: 10px 20px;
    font-size: 16px;
    cursor: pointer;
    border: none;
    border-radius: 5px;
    color: white;
  }
  .github-button {
    background-color: #24292e; /* GitHub dark background */
  }
  .github-button:hover {
    background-color: #444d56;
  }
  .google-button {
    background-color: #ff2f2f; /* Google blue */
  }
  .google-button:hover {
    background-color: #a01a1a;
  }
</style>
</head>
<body>
  <div>
    <h1>Welcome to OAuth App</h1>
    <h3>Log in with your favorite provider</h3>
    <div class="box">
      <button class="github-button" onclick="location.href='/auth/github'">Login with GitHub</button><br>
      <button class="google-button" onclick="location.href='/auth/google'">Login with Google</button><br>
    </div>
  </div>
</body>
</html>

```

Key Features:

- **Midnight blue background** with a centered white box
- Two styled buttons:
 - "Login with GitHub"
 - "Login with Google"
- When login is successful, the page:
 - Fetches session info
 - Shows a **browser notification**
 - Shows an **in-page alert**

- Removes `?login=success` from the URL
 - The logic is handled via JavaScript inside a `<script>` tag using `fetch('/session')`.
-

Step Three (Backend)

Backend (`server.js`)

- -File: `server.js`

This is the heart of the application. It handles:

- Routing
- Redirection to OAuth providers
- Handling callback after user authorizes
- Session status

```
const express = require('express');
const axios = require('axios');
const path = require('path');
require('dotenv').config();

const app = express();
const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'index.html'));
});

let userSession = {
  provider: null,
  access_token: null
};

// --- GitHub OAuth ---
app.get('/auth/github', (req, res) => {
  const githubAuthUrl = `https://github.com/login/oauth/authorize?client_id=${process.env.GITHUB_CLIENT_ID}&redirect`
  res.redirect(githubAuthUrl);
});

app.get('/auth/github/callback', async (req, res) => {
  const code = req.query.code;
  const tokenUrl = 'https://github.com/login/oauth/access_token';

  try {
    const response = await axios.post(tokenUrl, {
      client_id: process.env.GITHUB_CLIENT_ID,
      client_secret: process.env.GITHUB_CLIENT_SECRET,
      code: code,
      redirect_uri: process.env.GITHUB_CALLBACK_URL
    }, {
      headers: { Accept: 'application/json' }
    });

    userSession = {
      provider: 'GitHub',
      access_token: response.data.access_token
    };

    res.redirect('/?login=success');
  } catch (error) {
    console.error(error);
  }
});
```

```

    res.send('Error during GitHub OAuth');
  }
});

// --- Google OAuth ---
app.get('/auth/google', (req, res) => {
  console.log('>>> HIT /auth/google'); // <← our debug print
  const googleAuthUrl =
    'https://accounts.google.com/o/oauth2/v2/auth'
    + `?client_id=${process.env.GOOGLE_CLIENT_ID}`
    + `&redirect_uri=${process.env.GOOGLE_CALLBACK_URL}`
    + `&response_type=code`
    + `&scope=email%20profile`
    + `&prompt=consent`;
  res.redirect(googleAuthUrl);
});

app.get('/auth/google/callback', async (req, res) => {
  const code = req.query.code;
  const tokenUrl = 'https://oauth2.googleapis.com/token';

  try {
    const response = await axios.post(tokenUrl, {
      client_id: process.env.GOOGLE_CLIENT_ID,
      client_secret: process.env.GOOGLE_CLIENT_SECRET,
      code: code,
      grant_type: 'authorization_code',
      redirect_uri: process.env.GOOGLE_CALLBACK_URL
    });

    userSession = {
      provider: 'Google',
      access_token: response.data.access_token
    };

    res.redirect('/?login=success');
  } catch (error) {
    console.error(error);
    res.send('Error during Google OAuth');
  }
});

// --- Logout ---
app.get('/logout', (req, res) => {
  userSession = {
    provider: null,
    access_token: null
  };
  res.redirect('/');
});

// --- API to check session ---
app.get('/session', (req, res) => {
  res.json(userSession);
});

app.listen(PORT, () => {

```



```
console.log(`Server running at http://localhost:${PORT}`);
});
```

Breakdown of Logic:

1. Initial Page Load

```
app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'index.html'));
});
```

Serves the homepage when the user visits `http://localhost:3000` .

2. GitHub OAuth Flow

```
app.get('/auth/github', ...);
app.get('/auth/github/callback', async (req, res) => { ... });
```

- Redirects user to GitHub's authorization page.
 - Upon successful login, GitHub redirects to `/auth/github/callback` with a `code` .
 - This code is exchanged for an **access token** using `axios.post` .
 - Stores the token and provider name in a simple `userSession` object.
-

3. Google OAuth Flow

```
app.get('/auth/google', ...);
app.get('/auth/google/callback', async (req, res) => { ... });
```

- Similar to GitHub, it redirects to Google's OAuth URL.
 - Callback handler exchanges the code for a token.
 - Also stores token and provider name in `userSession` .
-

4. Session Status

```
app.get('/session', (req, res) => {
  res.json(userSession);
});
```

Returns the current login session info (which provider is logged in).

5. Logout

```
app.get('/logout', (req, res) => {
  userSession = { provider: null, access_token: null };
  res.redirect('/');
```

```
});
```

Clears the session manually and returns to homepage.

Step Four (.env)

Environment Configuration (`.env`)

File: `.env`

Contains all sensitive credentials for GitHub and Google:

```
# Github
GITHUB_CLIENT_ID=Ov23lisukAXr6Y1A6Joc
GITHUB_CLIENT_SECRET=dfebde5d0daf51785050b4455540ce25e89a6b9c
GITHUB_CALLBACK_URL=http://localhost:3000/auth/github/callback

# Google
GOOGLE_CLIENT_ID=985309607768-j83jjj1sr1s87637753ffmv42664lhg3.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSXP-T0wb6fsJ4ytAwd9W-xnGa499qKxo
GOOGLE_CALLBACK_URL=http://localhost:3000/auth/google/callback
```

These values are used in the backend to securely perform token exchanges

Project Execution Steps

1. Run the server:

```
node server.js
```

2. Go to browser:

Visit `http://localhost:3000`

3. Click a provider:

- Login via GitHub or Google
- Authorize the app

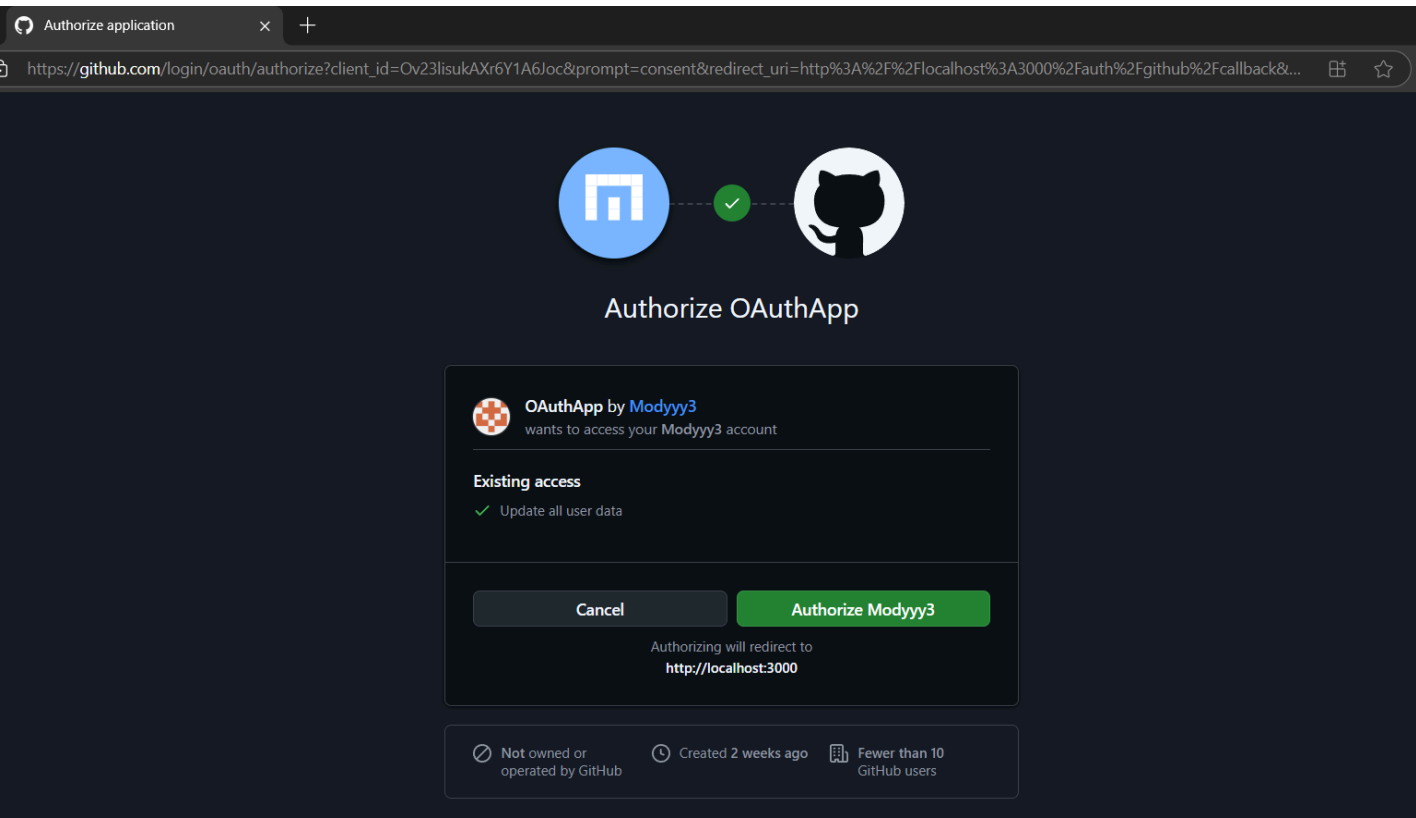
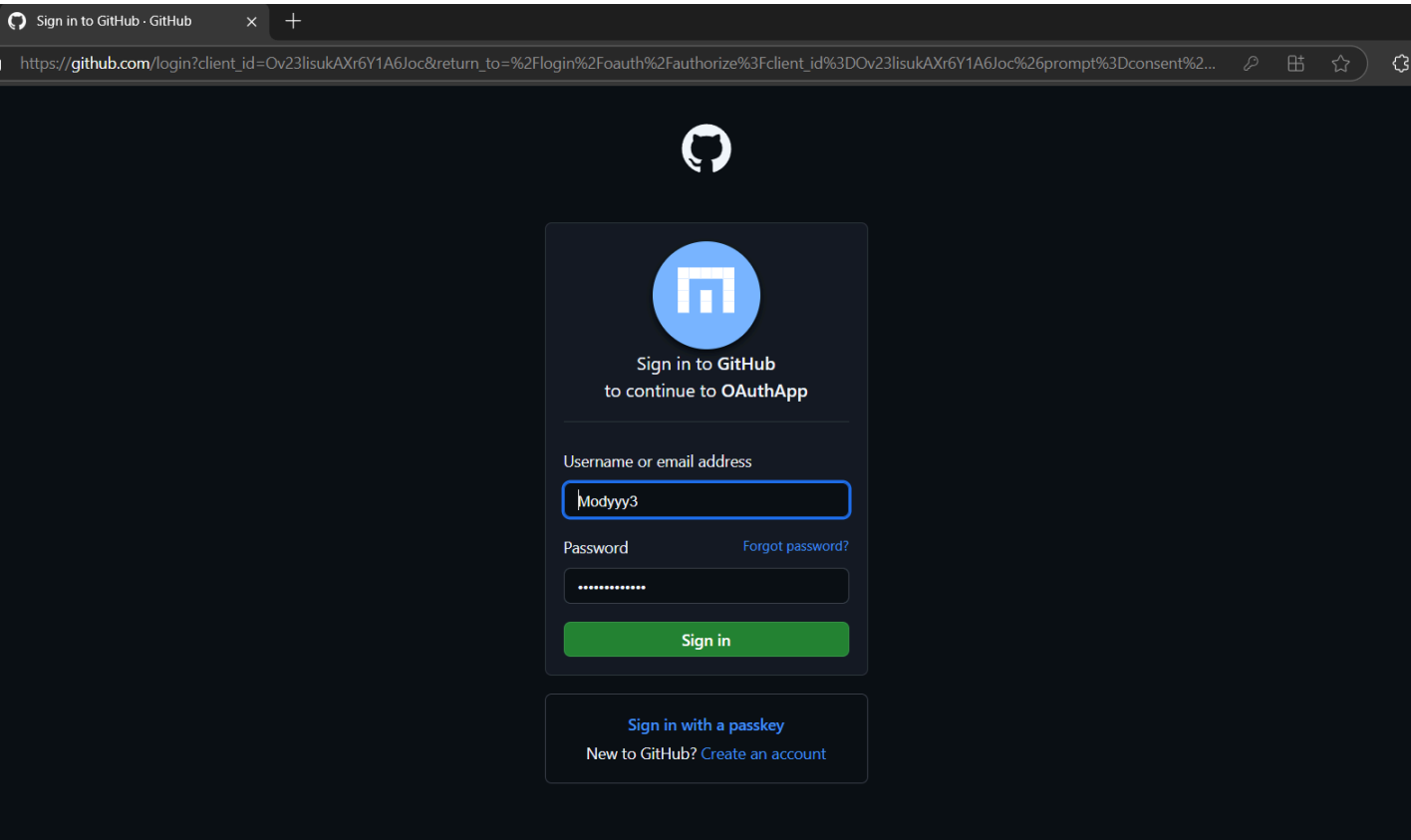
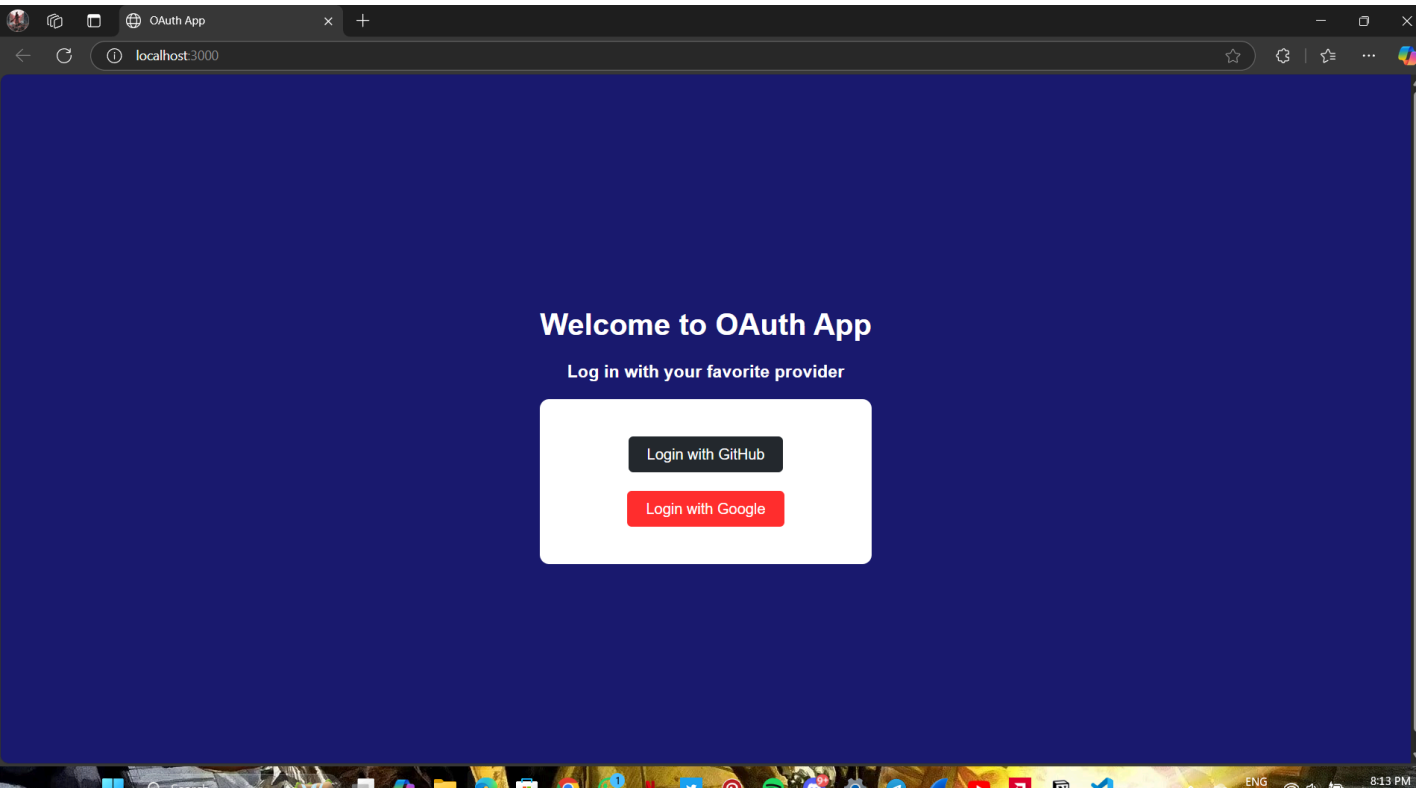
4. On success:

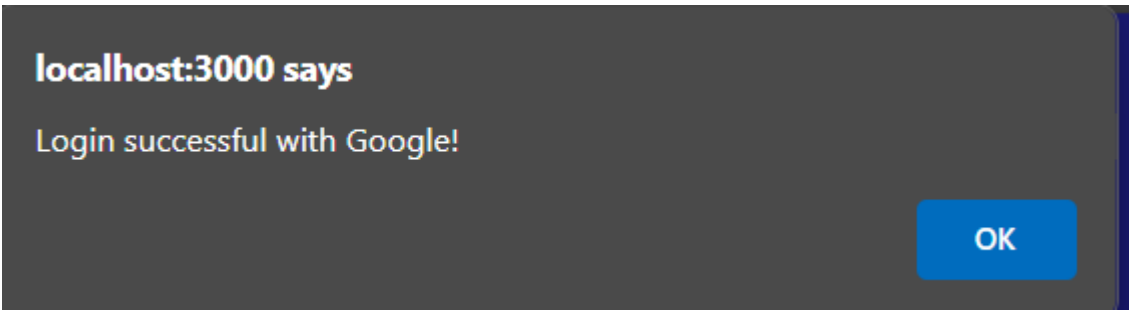
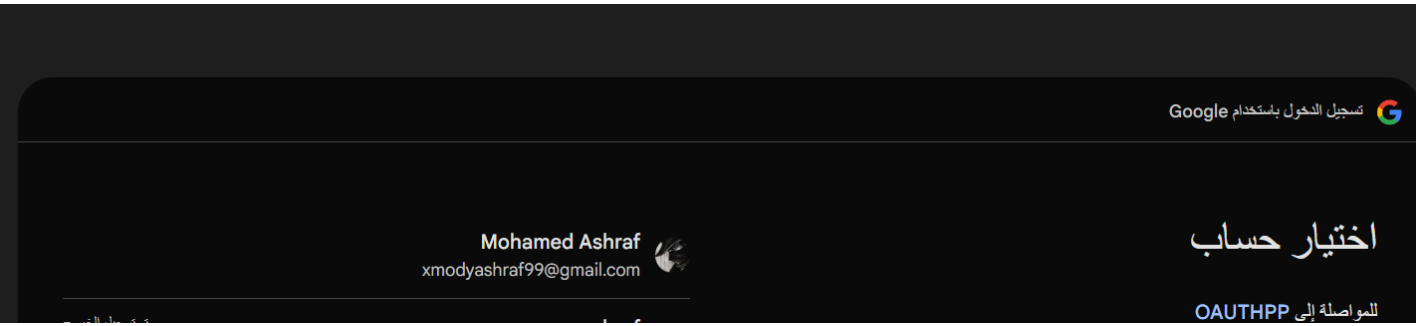
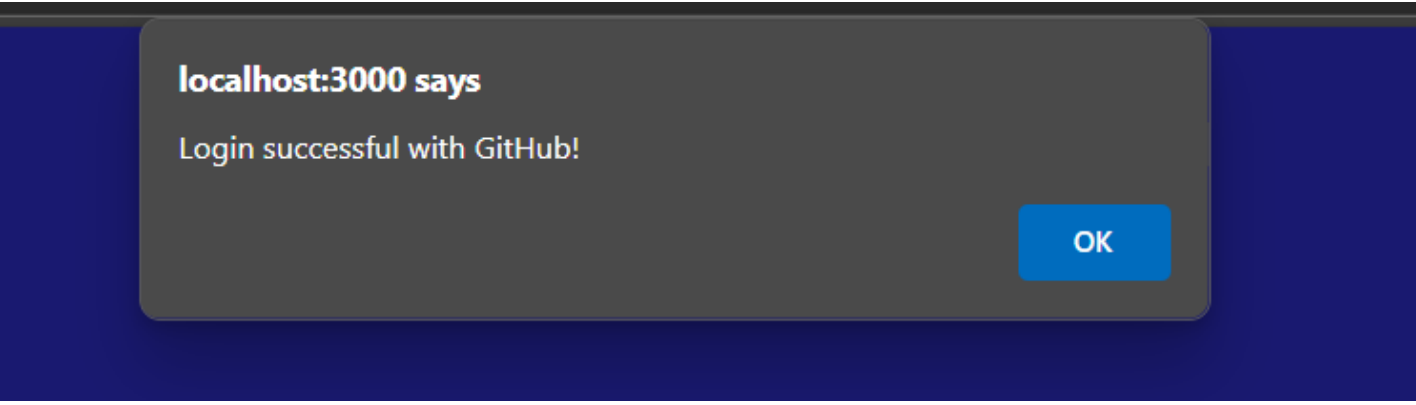
- Session saved in memory
- Browser alert + notification appears

App Run

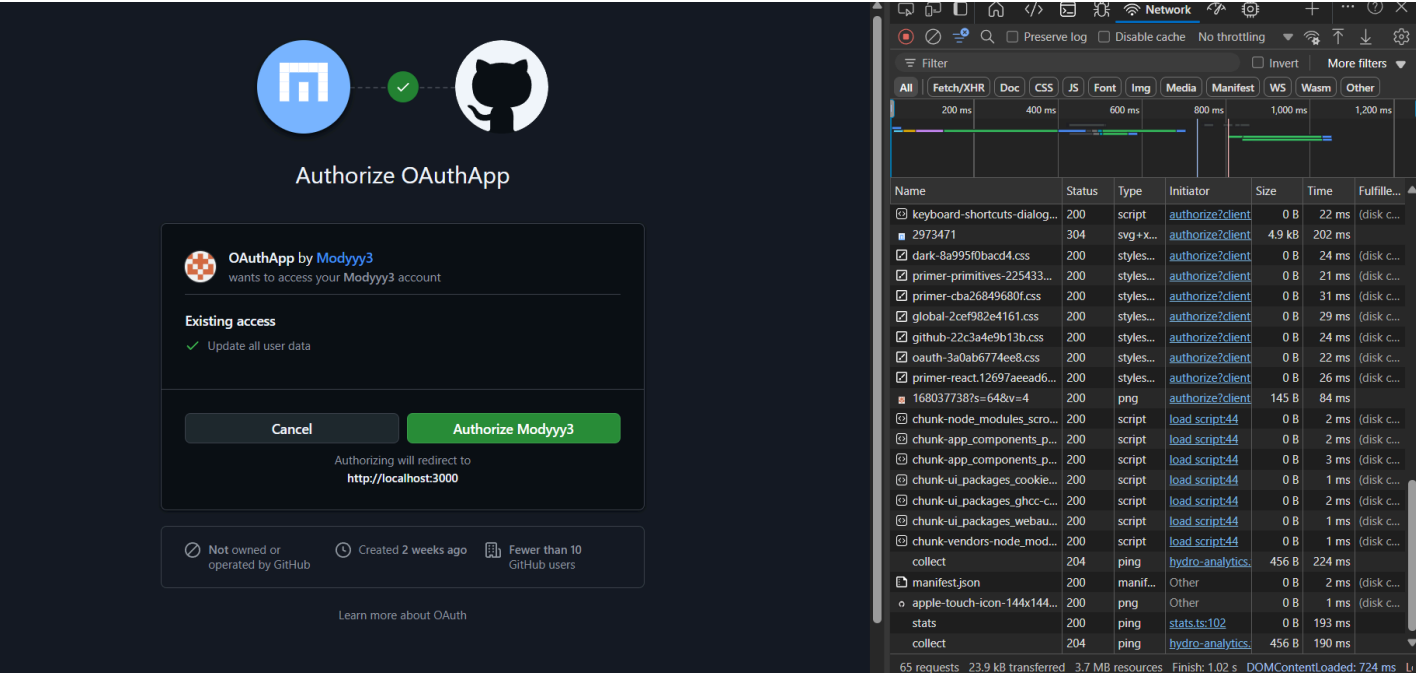
```
PS C:\Users\xmody\Desktop\oauth-project\oauth-project> node server.js
```

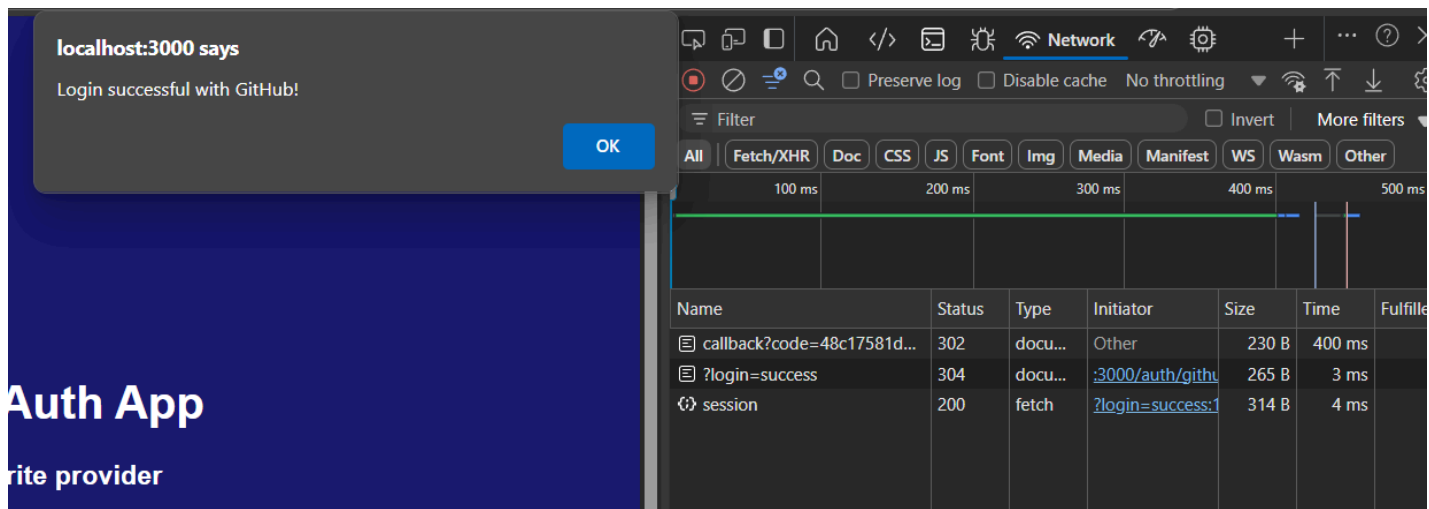
```
PS C:\Users\xmody\Desktop\oauth-project\oauth-project> node server.js
Server running at http://localhost:3000
```





- Network traffic during the run





-so what is difference between logging in with OAuth vs. without it?

- Login WITHOUT OAuth (Traditional Login)

- The **user creates an account** and **enters a username/password** directly into your app.
- Your app is responsible for:
 - Storing passwords securely (e.g., hashed in a database).
 - Creating session tokens.
 - Handling authentication and security.

Pros:

- Full control over user accounts and data.

Cons:

- You are responsible for **security** (password leaks, brute force attacks).
- Extra work for users (create and remember another password).
- Complex to build securely.

- Login WITH OAuth (Third-Party Login)

- The user **logs in through a trusted provider** like Google or GitHub.
- Your app **does NOT handle passwords**.
- Instead, the provider:
 - Authenticates the user.
 - Sends your app a **token** confirming the user's identity.

Pros:

- **More secure** (your app never sees passwords).
- **Faster and easier** login for users.
- You **offload authentication** to trusted providers (Google, GitHub, etc.).

Cons:

- You depend on external services being available.
- You need to register your app with each provider.

Feature	Traditional Login	OAuth Login (e.g., Google)
Handles user passwords	Your app	OAuth provider (e.g., Google)
Requires sign-up form	Yes	No (uses provider's account)
Security responsibility	You	Provider
User experience	Slower	Faster (1-click login)
Best for	Custom accounts	Social or developer logins

- This project demonstrates:

- A practical implementation of OAuth 2.0
- Handling third-party authorization securely
- Integrating frontend and backend session flow
- Real-world interaction with APIs like GitHub and Google
- Usage of environment variables for secure secret management

It simulates how many real-world applications (e.g., Zoom, Notion, Trello) authenticate users using providers like Google or GitHub without handling passwords directly.

- Conclusion

project successfully implements **OAuth 2.0** login with both GitHub and Google in a clean, functional web application. It helps users understand how to authorize securely without passwords and showcases a modern standard of web authentication.